Python Lab Manual & Solutions (UNIT 4)

Lab Exercise 11: Log File Reader Program

Problem Statement & Critical Thinking (CT)

Develop a Python program to read a log file, identify specific keywords such as ERROR, WARNING, and INFO, and summarize the number of occurrences of each category. The program analyzes log contents, handles missing or empty files gracefully using exception handling, and verifies results accurately.

Code

```
import os
from collections import Counter

def analyze_log_file(file_path):
    """
    Reads a system log file, categorizes entries by INFO, WARNING, and ERROR keywords,
    and returns a summary count of each log level.
    """
    keyword_counts = Counter({'INFO': 0, 'WARNING': 0, 'ERROR': 0})
    detailed_errors = []

    # Critical Thinking: Check file existence and non-empty status
    if not os.path.exists(file_path):
        print(f"[Error] File '{file_path}' does not exist.")
        return None

    if os.path.getsize(file_path) == 0:
        print(f"[Warning] File '{file_path}' is empty. No log entries to process.")
        return keyword_counts

    try:
        with open(file_path, 'r', encoding='utf-8') as file:
            for line_no, line in enumerate(file, 1):
                upper_line = line.upper()
                matched = False
                for kw in ['ERROR', 'WARNING', 'INFO']:
                    if kw in upper_line:
                        keyword_counts[kw] += 1
                        matched = True
                        if kw == 'ERROR':
                            detailed_errors.append((line_no, line.strip()))
                        break

        return keyword_counts, detailed_errors

    except Exception as e:
        print(f"[Error] Failed to read log file: {e}")
        return None

def generate_sample_log(filename="app_server.log"):
    """Generates a mock application log file for demonstration."""
    sample_logs = [
```

```
        "2026-11-02 08:30:15 INFO [Server] System initialized successfully.\n",
        "2026-11-02 08:31:02 INFO [DB] Connection established to PostgreSQL.\n",
        "2026-11-02 08:35:40 WARNING [Memory] High memory usage detected (82%).\n",
        "2026-11-02 08:40:12 ERROR [Auth] Failed login attempt for user 'admin' from IP
192.168.1.45.\n",
        "2026-11-02 08:42:00 INFO [Cache] Cache invalidated for session tokens.\n",
        "2026-11-02 08:50:22 WARNING [Disk] Disk storage space below 15%.\n",
        "2026-11-02 09:01:10 ERROR [DB] Timeout connecting to replica database.\n",
        "2026-11-02 09:05:00 ERROR [API] 500 Internal Server Error on endpoint /api/v1/
checkout.\n"
    ]
    with open(filename, 'w', encoding='utf-8') as f:
        f.writelines(sample_logs)

if __name__ == "__main__":
    log_filename = "app_server.log"
    generate_sample_log(log_filename)

    counts, errors = analyze_log_file(log_filename)

    if counts is not None:
        print("=" * 60)
        print(f"{'LOG LEVEL':<15} | {'OCCURRENCE COUNT':<20}")
        print("=" * 60)
        for level, count in counts.items():
            print(f"{level:<15} | {count:<20}")
        print("=" * 60)

        print("\n[Detailed Error Incidents]")
        for line_no, msg in errors:
            print(f" Line {line_no:02d}: {msg}")
```

Output

```
============================================================
LOG LEVEL       | OCCURRENCE COUNT
============================================================
INFO            | 3
WARNING         | 2
ERROR           | 3
============================================================

[Detailed Error Incidents]
 Line 04: 2026-11-02 08:40:12 ERROR [Auth] Failed login attempt for user 'admin' from IP
192.168.1.45.
 Line 07: 2026-11-02 09:01:10 ERROR [DB] Timeout connecting to replica database.
 Line 08: 2026-11-02 09:05:00 ERROR [API] 500 Internal Server Error on endpoint /api/v1/
checkout.
```

Real-World Application & Analysis (RWE)

Application in Real World: Log analysis is foundational to DevOps, cybersecurity, and system administration. Automated log parsing allows engineers to monitor cloud servers, detect security threats (such as brute-force authentication failures), and identify critical service outages instantly without manually sifting through gigabytes of raw log files.

Lab Exercise 12: Student Data Manager Using Files

Problem Statement & Critical Thinking (CT)

Develop a Python program to store, retrieve, search, update, and display student records using file operations. Design record-management logic using structured text files, validate input values (e.g., USN, GPA), and test retrieval/updates.

Code:

```
import os

class StudentDataManager:
    def __init__(self, filepath="students_db.txt"):
        self.filepath = filepath

    def _read_all(self):
        records = {}
        if not os.path.exists(self.filepath):
            return records
        with open(self.filepath, 'r', encoding='utf-8') as f:
            for line in f:
                line = line.strip()
                if line:
                    parts = line.split('|')
                    if len(parts) == 4:
                        usn, name, dept, gpa = parts
                        records[usn] = {"name": name, "dept": dept, "gpa": float(gpa)}
        return records

    def _write_all(self, records):
        with open(self.filepath, 'w', encoding='utf-8') as f:
            for usn, info in records.items():
                f.write(f"{usn}|{info['name']}|{info['dept']}|{info['gpa']:.2f}\n")

    def add_student(self, usn, name, dept, gpa):
        records = self._read_all()
        if usn in records:
            print(f"[Error] Student with USN '{usn}' already exists.")
            return False
        records[usn] = {"name": name, "dept": dept, "gpa": float(gpa)}
        self._write_all(records)
        print(f"[Success] Student '{name}' (USN: {usn}) added successfully.")
        return True

    def search_student(self, usn):
        records = self._read_all()
        if usn in records:
            info = records[usn]
            print(f"\n[Record Found] USN: {usn} | Name: {info['name']} | Dept:
{info['dept']} | GPA: {info['gpa']:.2f}")
            return info
        print(f"\n[NotFound] No student found with USN: {usn}")
        return None

    def update_gpa(self, usn, new_gpa):
```

```
        records = self._read_all()
        if usn not in records:
            print(f"[Error] Cannot update. Student '{usn}' not found.")
            return False
        old_gpa = records[usn]['gpa']
        records[usn]['gpa'] = float(new_gpa)
        self._write_all(records)
        print(f"[Success] Updated {records[usn]['name']}'s GPA: {old_gpa:.2f} ->
{new_gpa:.2f}")
        return True

    def display_all(self):
        records = self._read_all()
        print("\n" + "=" * 65)
        print(f"{'USN':<15} | {'Student Name':<20} | {'Dept':<10} | {'GPA':<6}")
        print("=" * 65)
        for usn, info in records.items():
            print(f"{usn:<15} | {info['name']:<20} | {info['dept']:<10} |
{info['gpa']:<6.2f}")
        print("=" * 65)

if __name__ == "__main__":
    db = StudentDataManager("students_db.txt")

    # 1. Add records
    db.add_student("25BTRAD108", "Shriharan K", "CSE-AIDE", 9.45)
    db.add_student("25BTRAD109", "Aarav Sharma", "CSE-AIDE", 8.80)
    db.add_student("25BTRAD110", "Priya Patel", "ECE", 9.15)

    # 2. Display all records
    db.display_all()

    # 3. Search for a record
    db.search_student("25BTRAD108")

    # 4. Update a record
    db.update_gpa("25BTRAD109", 9.05)

    # 5. Display updated records
    db.display_all()
```

Output

```
[Success] Student 'Shriharan K' (USN: 25BTRAD108) added successfully.
[Success] Student 'Aarav Sharma' (USN: 25BTRAD109) added successfully.
[Success] Student 'Priya Patel' (USN: 25BTRAD110) added successfully.

================================================================
USN             | Student Name          | Dept       | GPA
================================================================
25BTRAD108      | Shriharan K           | CSE-AIDE   | 9.45
25BTRAD109      | Aarav Sharma          | CSE-AIDE   | 8.80
25BTRAD110      | Priya Patel           | ECE        | 9.15
================================================================

[Record Found] USN: 25BTRAD108 | Name: Shriharan K | Dept: CSE-AIDE | GPA: 9.45
[Success] Updated Aarav Sharma's GPA: 8.80 -> 9.05

================================================================
USN             | Student Name          | Dept       | GPA
================================================================
25BTRAD108      | Shriharan K           | CSE-AIDE   | 9.45
25BTRAD109      | Aarav Sharma          | CSE-AIDE   | 9.05
25BTRAD110      | Priya Patel           | ECE        | 9.15
================================================================
```

Real-World Application & Analysis (RWE)

Application in Real World: Educational institutions and academic management systems require persistent record storage for grading, enrollment tracking, and transcripts. Implementing file-based databases with CRUD (Create, Read, Update, Delete) operations forms the underlying foundation for backend database management systems (DBMS) and administrative record archives.

Lab Exercise 13: CSV-Based Payroll Management System

Problem Statement & Critical Thinking (CT)

Develop a Python program to read employee records from a CSV file, calculate gross salary and applicable deductions (such as Tax and Provident Fund), and generate structured salary summaries. Process CSV datasets dynamically and verify accuracy.

Code:

```
import csv
import os

class PayrollSystem:
    def __init__(self, tax_rate=0.12, pf_rate=0.05):
        self.tax_rate = tax_rate  # 12% Income Tax
        self.pf_rate = pf_rate    # 5% Provident Fund

    def process_payroll(self, input_csv, output_csv):
        """Reads employee data from CSV, computes gross/net salaries, and outputs summary
CSV."""
        if not os.path.exists(input_csv):
            print(f"[Error] Input file '{input_csv}' not found.")
            return
```

```
        processed_data = []
        total_payout = 0.0

        with open(input_csv, mode='r', encoding='utf-8') as file:
            reader = csv.DictReader(file)
            for row in reader:
                emp_id = row['EmpID']
                name = row['Name']
                basic = float(row['BasicSalary'])
                allowance = float(row['Allowance'])

                # Salary Computations
                gross_salary = basic + allowance
                tax = gross_salary * self.tax_rate
                pf = gross_salary * self.pf_rate
                total_deductions = tax + pf
                net_salary = gross_salary - total_deductions

                total_payout += net_salary

                processed_data.append({
                    'EmpID': emp_id,
                    'Name': name,
                    'Basic': basic,
                    'Allowance': allowance,
                    'Gross': gross_salary,
                    'Deductions': total_deductions,
                    'NetSalary': net_salary
                })

        # Write to summary CSV
        fieldnames = ['EmpID', 'Name', 'Basic', 'Allowance', 'Gross', 'Deductions',
'NetSalary']
        with open(output_csv, mode='w', newline='', encoding='utf-8') as file:
            writer = csv.DictWriter(file, fieldnames=fieldnames)
            writer.writeheader()
            for row in processed_data:
                writer.writerow(row)

        self.display_summary(processed_data, total_payout)

    def display_summary(self, processed_data, total_payout):
        print("\n" + "=" * 78)
        print(f"{'EmpID':<8} | {'Employee Name':<18} | {'Gross ($)':<10} | {'Deductions
($)':<14} | {'Net Salary ($)':<12}")
        print("=" * 78)
        for r in processed_data:
            print(f"{r['EmpID']:<8} | {r['Name']:<18} | {r['Gross']:<10.2f} |
{r['Deductions']:<14.2f} | ${r['NetSalary']:<11.2f}")
        print("=" * 78)
        print(f"{'TOTAL ORGANIZATIONAL PAYOUT':<56} | ${total_payout:<12.2f}")
        print("=" * 78)

def setup_sample_employee_csv(filename="employees.csv"):
    headers = ["EmpID", "Name", "BasicSalary", "Allowance"]
    rows = [
```

```
        ["EMP001", "John Doe", "45000", "5000"],
        ["EMP002", "Jane Smith", "60000", "8000"],
        ["EMP003", "Robert Johnson", "35000", "3000"],
        ["EMP004", "Emily Davis", "52000", "6000"]
    ]
    with open(filename, 'w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerow(headers)
        writer.writerows(rows)

if __name__ == "__main__":
    input_file = "employees.csv"
    output_file = "payroll_summary.csv"

    setup_sample_employee_csv(input_file)
    payroll = PayrollSystem(tax_rate=0.12, pf_rate=0.05)
    payroll.process_payroll(input_file, output_file)
```

Output

```
================================================================================
EmpID    | Employee Name      | Gross ($)  | Deductions ($) | Net Salary ($)
================================================================================
EMP001   | John Doe           | 50000.00   | 8500.00        | $41500.00
EMP002   | Jane Smith         | 68000.00   | 11560.00       | $56440.00
EMP003   | Robert Johnson     | 38000.00   | 6460.00        | $31540.00
EMP004   | Emily Davis        | 58000.00   | 9860.00        | $48140.00
================================================================================
TOTAL ORGANIZATIONAL PAYOUT                                | $177620.00
================================================================================
```

Real-World Application & Analysis (RWE)

Application in Real World: CSV processing forms the backbone of HR and accounting software (e.g., ADP, QuickBooks, SAP). Automating payroll computations via file automation reduces human error, guarantees exact statutory tax deductions, and generates audit-ready financial distribution reports rapidly for corporate finance departments.